@import url('https://fonts.googleapis.com/css2?family=Roboto+Flex:opsz,wdth,wght,GRAD@8..144,25..151,100..1000,-200..150&display=swap'); @page { size: A4; margin: 20mm; @bottom-right { content: counter(page); font-family: 'Roboto Flex', sans-serif; font-size: 9px; color: #757688; } } body { font-family: 'Roboto Flex', sans-serif; color: #1a1b24; background-color: #ffffff; line-height: 1.6; font-size: 14px; margin: 0; padding: 0; } h1, h2, h3, h4, h5, h6 { color: #213be7; font-family: 'Roboto Flex', sans-serif; font-weight: 500; margin-top: 1.5em; margin-bottom: 0.5em; page-break-after: avoid; } h1 { font-size: 28px; border-bottom: 2px solid #213be7; padding-bottom: 0.3em; margin-top: 0; } h2 { font-size: 20px; border-bottom: 1px solid #c5c5d9; padding-bottom: 0.2em; } h3 { font-size: 16px; } p, li { color: #444656; margin-bottom: 1em; } ul, ol { margin-bottom: 1.5em; padding-left: 20px; } li { margin-bottom: 0.5em; } table { width: 100%; border-collapse: collapse; margin-bottom: 1.5em; page-break-inside: avoid; font-size: 13px; } th, td { border: 1px solid #c5c5d9; padding: 8px 12px; text-align: left; } th { background-color: #eeecfa; font-weight: 600; color: #1a1b24; } tr:nth-child(even) { background-color: #f4f2ff; } pre { background-color: #eeecfa; border: 1px solid #c5c5d9; border-radius: 8px; padding: 12px; overflow-x: auto; font-family: monospace; font-size: 13px; margin-bottom: 1.5em; page-break-inside: avoid; } code { font-family: monospace; background-color: #eeecfa; color: #1a1b24; padding: 2px 4px; border-radius: 4px; font-size: 13px; } pre code { background-color: transparent; padding: 0; border-radius: 0; color: inherit; } blockquote { border-left: 4px solid #973500; margin: 1.5em 0; color: #973500; background-color: #ffdbce; border-radius: 0 8px 8px 0; padding: 12px 16px; } blockquote p { margin: 0; color: #370e00; font-weight: 500; } /* Mermaid styles */ .mermaid { display: flex; justify-content: center; margin: 2em 0; page-break-inside: avoid; } /* Cover and header rules */ hr { border: 0; border-top: 1px solid #c5c5d9; margin: 2em 0; }

Daffodil Intelligence (DI) — Project Documentation

Professional & Formal Technical Specifications and Architectural Design

---

1. Executive Summary

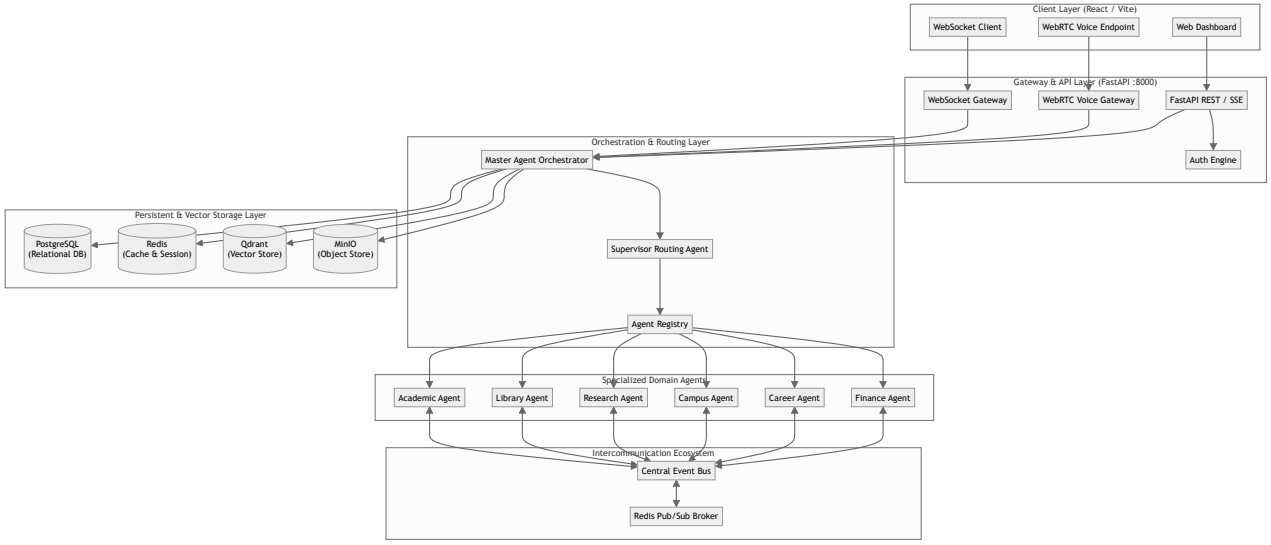
Daffodil Intelligence (DI) is a 360° Multi-Agent Artificial Intelligence Operating System designed for academic institutions, specifically tailored for university students, faculty members, and administrative staff. By transitioning from a basic reactive virtual assistant to a proactive multi-agent orchestration platform, Daffodil Intelligence integrates academic databases, campus transportation, research directories, real-time voice endpoints (WebRTC), and Model Context Protocol (MCP) tool bindings.

This documentation serves as the comprehensive source of truth for the Daffodil Intelligence architecture, covering its frontend web interface, reactive routing engine, agent pool, and external service gateways. Furthermore, it details the formal design specifications for two core platform extensions: 1. **The Intercommunication Ecosystem (Agent-to-Agent Event Bus)** 2. **The Research Community Hub**

---

2. System Architecture & Component Topology

Daffodil Intelligence employs a decoupled, multi-tier system architecture designed for high availability, low-latency agent routing, and secure vector search capabilities.



Component Details

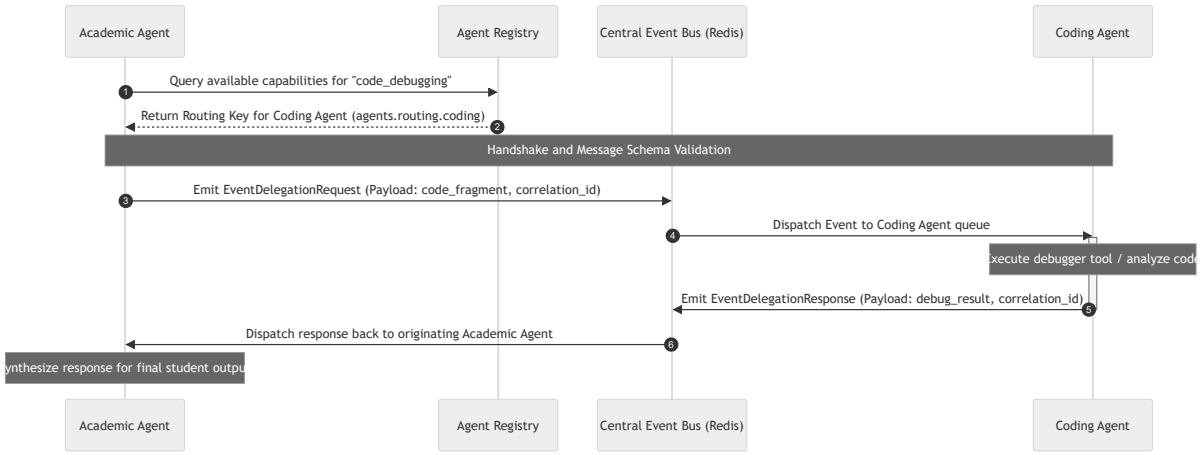
1. **Frontend Web Dashboard (`apps/web`)**: Implemented in React, Vite, MUI (Material-UI), and Zustand. It operates a real-time conversation screen, dynamic agent-status feeds, and live voice call integrations.
2. **Backend API Gateway (`apps/api`)**: Built on FastAPI. Serves as the central router for HTTP, Server-Sent Events (SSE), and WebSocket traffic. It manages connection sessions and authenticates requests via JWT tokens.
3. **Orchestrator Engine (`apps/api/app/runtime`)**: Uses a dual routing strategy:
4. *Supervisor Routing*: Uses semantic classification and keyword analysis to distribute incoming user messages to dedicated, specialized agents.
5. *LangGraph Workflows*: Models multi-step, parallel agent execution paths as directed acyclic graphs (DAGs).
6. **Tool Registry & MCP Server (`packages/mcp`)**: FastMCP-compliant server that exposes domain tools to agents, offering rate-limiting, permissions, and result validation.
7. **Data Store Layer**:
8. **PostgreSQL**: Stores persistent relational entities (Users, Credentials, Chat History, Agent Configurations).
9. **Redis**: Serves as a high-performance memory cache, WebSocket pub/sub engine, and Celery task broker.
10. **Qdrant**: Manages vector-embeddings for semantic RAG search.

11. **MinIO**: Handles object storage for PDFs, images, and audio attachments.

---

3. Core Subfeature: Intercommunication Ecosystem

The **Intercommunication Ecosystem** introduces a decentralized, event-driven agent-to-agent (A2A) networking mesh. It allows independent agents to discover, negotiate, and delegate sub-tasks dynamically without manual supervisor intervention.



3.1 Architecture Specifications

- **Central Event Broker**: Driven by Redis Pub/Sub channels. Sub-queues are established for each specialized agent class using patterns such as `di.agents.<agent_name>.inbound` and `di.agents.<agent_name>.outbound`.
- **Agent Registry**: Active agents register their status (`active`, `busy`, `ready`), supported tools, and semantic capability schemas to Redis with a 60-second TTL heartbeat. This metadata is parsed dynamically during routing decisions.
- **Dynamic Event Routing**: When an agent detects a query component it cannot handle (e.g., the Academic Agent parsing a request that demands code writing), it publishes a message containing a standard routing header to target appropriate secondary agents.

3.2 Protocol & Schema Definition

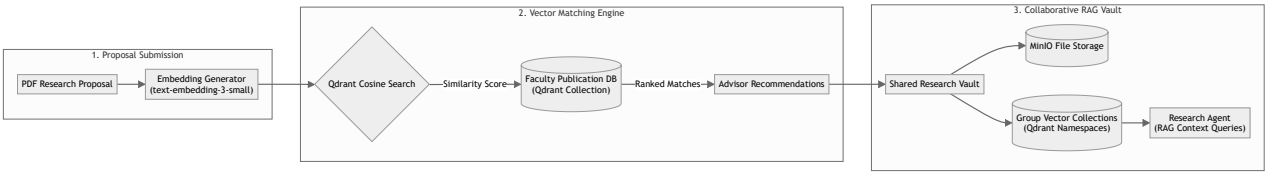
To prevent type-drift, all inter-agent communication must conform to the **CloudEvents v1.0.2** specifications wrapped in a custom JSON schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "AgentIntercomEvent",
  "type": "object",
  "properties": {
    "id": { "type": "string", "format": "uuid" },
    "source": { "type": "string", "example": "di.agents.academic" },
    "type": { "type": "string", "example": "di.events.delegation.request" },
    "specversion": { "type": "string", "const": "1.0.2" },
    "time": { "type": "string", "format": "date-time" },
    "correlation_id": { "type": "string", "format": "uuid" },
    "session_id": { "type": "integer" },
    "destination": { "type": "string", "example": "di.agents.coding" },
    "data": {
      "type": "object",
      "properties": {
        "required_action": { "type": "string" },
        "payload": { "type": "object" },
        "timeout_ms": { "type": "integer", "default": 5000 }
      },
      "required": ["required_action", "payload"]
    }
  },
  "required": ["id", "source", "type", "specversion", "time", "correlation_id", "session_id", "destination", "data"]
}
```

---

4. Core Subfeature: Research Community Platform

The **Research Community Platform** bridges academic research, student initiatives, and thesis supervisions into a collaborative network powered by vector matching and semantic RAG vaults.



4.1 Advisor Matchmaking System

This system addresses a common university friction point: matching graduate and undergraduate students with prospective thesis advisors. - **Data Ingestion**: Faculty publication records (abstracts, journal articles, active projects) are automatically scraped and synchronized using the University HR/Library APIs. - **Embedding & Storage**: Document records are processed using `text-embedding-3-small` and saved in a dedicated `faculty_publications` collection in Qdrant. - **Matching Pipeline**: When a student submits a thesis draft or proposal abstract, DI computes its vector representation, executes a cosine similarity search against `faculty_publications`, and returns the top 3 faculty profiles with detailed matching scores, publication citations, and research overlap highlights.

4.2 Shared Knowledge Vaults (RAG)

Designed for research groups, thesis circles, and laboratory courses: - **Joint Reading Vaults**: Research groups can instantiate shared folders via the frontend. Documents uploaded to these folders are persisted in MinIO bucket locations unique to that group. - **Dynamic Ingestion Pipeline**: Worker nodes chunk the documents and write embeddings to a collection in Qdrant namespace-isolated for the group: `group_<group_id>_vault`. - **Collaborative RAG queries**: Group members query the collective knowledge vault. The `Research Agent` retrieves matching vectors strictly from the namespace-isolated index, preventing cross-group data leakage.

4.3 Thesis Progress & Automated Milestones Tracker

Automates thesis management workflows: - **Milestone Management**: Tracks student progress across typical academic milestones: *Proposal Submission, Mid-term Evaluation, Manuscript Draft, Final Defense*. - **Automatic Compliance Checks**: Integrates citation validation routines verifying manuscript references against standard guidelines (APA, MLA, IEEE) using external bibliographic endpoints. - **Notification Routing**: When milestone status shifts, updates are sent dynamically over the *Intercommunication Ecosystem* to the `Notification Agent`, which dispatches notifications to students and faculty through WebSockets, Telegram channels, or WhatsApp.

4.4 Research Hub & Publication Feed

The **Research Hub** serves as a public directory where students and faculty members showcase their research projects and publications. - **Reference-Link Ingestion**: Rather than hosting entire text documents within the repository database, users add reference URLs (e.g., DOI, arXiv, ResearchGate, or institutional links). - **Publication Card Renderer**: Upon submission, the gateway fetches bibliographic metadata and generates a dynamic "Publication Card" displayed on the user's public profile and the global trending directory. - **Trending & Engagement Indexing**: A background worker tabulates weekly views, citations, and semantic query matches to maintain a "Trending Research" section, highlighting popular scientific investigations inside the university network.

4.5 Specialized Research Sub-Agents & Dynamic Ingestion

To support rigorous academic reviews, the research platform employs two autonomous sub-agents: - **AI Citation & Integrity Verification Agent**: Performs structural checks to detect generative AI fingerprints in uploaded drafts, flagging synthetic anomalies, verifying bibliography authenticity, and outputting an AI-similarity score. - **Human-like Research Agent (Literature Review Specialist)**: Dynamically parses search databases, clusters papers, builds citation graphs, synthesizes contradictions or alignments in literature, and drafts summaries formatted according to specific styling guidelines. - **Dynamic Ingestion to Agent Skill Memory**: When a student or teacher adds a research link, a worker downloads, extracts, and vectorizes the document in the background. The resulting embeddings are dynamically loaded into the active context memory of the student's personal `Research Agent`. This allows the student to immediately query, dissect, and converse with the publication using natural language.

---

5. Dynamic Agent Spawning & Skill Hub System

Daffodil Intelligence features a dynamic execution layer allowing runtime modification of agent configurations, prompt behaviors, and capability bindings.

5.1 Programmatic Agent Spawning & Dynamic Sub-Agents

Rather than relying solely on the 18 pre-defined static agents, the `Master Agent Orchestrator` can programmatically spawn specialized sub-agents on demand: - **Runtime Compilation**: If a complex user goal is decomposed by the `Planning Agent` into highly specialized subtasks (e.g., analyzing a specific legacy COBOL file), the orchestrator dynamically creates an instance of a new sub-agent, compiles a target system prompt, binds requested tools, and registers it to the active session. - **Context Jailing**: Spawned sub-agents run in isolated execution blocks, inheriting only the necessary subset of conversation history and system parameters.

5.2 Installable Agent Skill Hub (App Store Model)

The **Skill Hub** serves as a modular registry (analogous to a mobile app store) where custom tools, APIs, and computational libraries are organized as installable modules. - **App-Store Style Management**: The React frontend provides a visual directory of all available skills (e.g., a "Bengali NLP Parser", "Financial Fee Calculator", or "VS Code Workspace Integration"). - **One-Click Binding**: When a user or supervisor agent clicks the "Install" button on a skill, the backend dynamically loads the skill's Python runtime package and binds its functional entry point to the target agent. The agent instantly updates its system schema, declaring the new tool to the LLM during the next chat turn.

5.3 Own MCP Builder & Dynamic Document-to-MCP Server

To enable developers and agents to interface with external workflows, the platform provides: - **Own MCP Builder**: A developer utility that programmatically translates standard Python functions and API paths into a fully compliant Model Context Protocol (MCP) server container, generating dynamic schemas, validation rules, and endpoint bindings automatically. - **Dynamic Document-to-MCP Server**: Integrations that dynamically map uploaded files (PDFs, Word Documents, Excel sheets, text files) into a transient MCP server. Once a file is uploaded, the platform launches a dedicated MCP interface around it, exposing document-specific tools (e.g., `search_sections`, `retrieve_table_data`, `summarize_pages`). The target LLM interacts with the file directly via structured protocol tool-calls rather than consuming the raw, token-heavy text context.

---

6. Agent Ecosystem & Catalog

Daffodil Intelligence orchestrates 18 specialized domain agents:

#	Agent Class	System Identifier	Core Capabilities
1	**Supervisor Agent**	`supervisor`	Orchestration, intent classification, multi-agent dispatch, response synthesis.
2	**Academic Agent**	`academic`	CGPA calculation, course registration advice, syllabus reviews, semester GPA audits.
3	**Library Agent**	`library`	Book searches, electronic resources retrieval, physical book seat reservations.
4	**Campus Services Agent**	`campus`	Room booking management, campus announcements routing, facilities statuses.
5	**Career Advisor Agent**	`career`	Internship tracking, resume parsing & scoring, career roadmaps creation.
6	**Coding Agent**	`coding`	Code generation, syntactic debugging, technical interview training.
7	**Research Agent**	`research`	Literature discovery, citation styling, abstract reviews, thesis brainstorming.
8	**Wellness Agent**	`wellness`	Student stress management protocols, cognitive guidelines, academic study habits.
9	**Finance Agent**	`finance`	Fee structure auditing, scholarship waiver analysis, installment calendar tracking.
10	**Vision Agent**	`vision`	Document OCR, user interface screenshots parsing, charts/graphs interpretations.
11	**Voice Agent**	`voice`	WebRTC audio synthesis, voice message translation, transcription pipeline.
12	**Exam Agent**	`exam`	Midterm/Final schedules audit, seat planning locations, attendance tracking.
13	**Hostel Agent**	`hostel`	Residence hall checks, service requests, room vacancy management.
14	**Transport Agent**	`transport`	University shuttle tracking, route timelines, special event transport bookings.
15	**Event Agent**	`event`	Club activity coordination, workshop notifications, campus calendars.
16	**Complaint Agent**	`complaint`	Student grievance collection, ticket queue management, support escalation.
17	**Notification Agent**	`notification`	Multi-channel notifications routing (Telegram, email, WebSocket alerts).
18	**Fee Agent**	`fee_agent`	Tuition statement audit, billing disputes, waivers verification.

---

7. Database Schema Design (Core Entities)

Below is the PostgreSQL schema defining users, session records, academic tracking, and research vaults.

```
-- Core User Schema
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    student_id VARCHAR(50) UNIQUE,
    department VARCHAR(100),
    current_semester INT,
    role VARCHAR(20) DEFAULT 'student' CHECK (role IN ('student', 'faculty', 'admin')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Academic Profiles
CREATE TABLE student_academic_profiles (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    cgpa NUMERIC(3, 2) NOT NULL,
    completed_credits INT NOT NULL,
    probation_status BOOLEAN DEFAULT FALSE,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Research Communities
CREATE TABLE research_groups (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    topic VARCHAR(255),
    created_by INT REFERENCES users(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Research Group Memberships
CREATE TABLE research_group_members (
    group_id INT REFERENCES research_groups(id) ON DELETE CASCADE,
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(50) DEFAULT 'member' CHECK (role IN ('lead', 'member', 'advisor')),
    PRIMARY KEY (group_id, user_id)
);

-- Collaborative Knowledge Vault Files
CREATE TABLE knowledge_vault_documents (
    id SERIAL PRIMARY KEY,
    group_id INT REFERENCES research_groups(id) ON DELETE CASCADE,
    uploaded_by INT REFERENCES users(id),
    file_name VARCHAR(255) NOT NULL,
    minio_object_key VARCHAR(512) NOT NULL,
    qdrant_collection_name VARCHAR(255) NOT NULL,
    uploaded_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Thesis Progress Tracking
CREATE TABLE thesis_milestones (
    id SERIAL PRIMARY KEY,
    student_id INT REFERENCES users(id) ON DELETE CASCADE,
    advisor_id INT REFERENCES users(id) ON DELETE SET NULL,
    title VARCHAR(512) NOT NULL,
    abstract TEXT,
    milestone_stage VARCHAR(50) DEFAULT 'proposal' CHECK (milestone_stage IN ('proposal', 'midterm', 'draft', 'defense_prep', 'complete
    citation_standard VARCHAR(20) DEFAULT 'APA',
    compliance_score INT,
```

```
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

---

8. API Routing & Endpoints

The backend exposes RESTful routers alongside real-time channels:

7.1 Authentication (`/api/auth`)

- `POST /register`: Registers student/faculty profiles.
- `POST /login`: Validates password and generates JWT.
- `POST /refresh`: Issues a replacement token using a refresh token.
- `GET /google`, `/microsoft`, `/github`: Third-party identity hooks.

7.2 Orchestrated Chats (`/api/agents`)

- `POST /chat`: Simple request-response interaction routing through the Supervisor.
- `POST /chat/stream`: Stream responses token-by-token (Server-Sent Events).
- `POST /tools/execute`: Executes registered tools directly.
- `POST /vision/analyze`: Ingests base64/binary screenshot feeds for UI element localization.

7.3 Real-time & WebSocket Routes (`/ws/{user_id}`)

- Establishes a persistent socket stream routing instant messages, system notifications, and agent activity heartbeats.

---

9. Development & Installation Guide

Prerequisites

- Python 3.11+
- Node.js 18+ (pnpm package manager)
- Docker & Docker Compose

Step 1: Infrastructure Initialization

Clone the repository, configure the environment, and spin up the local backing services:

```
# Copy settings templates
cp .env.example .env
cp apps/api/.env.example apps/api/.env

# Spin up Postgres, Redis, MinIO, and Qdrant
docker compose up -d
```

Step 2: Database Initialization & Migrations

Ensure the database contains the correct schemas and tables:

```
cd apps/api
../../.venv/bin/alembic upgrade head
../../.venv/bin/python scripts/archive/migrate_json_to_postgres.py
```

Step 3: Run the Application Core

You can run the full ecosystem in development mode or launch components individually:

```
# From workspace root
pnpm install
pnpm dev

# Or start services separately
pnpm docker:infra # Backing resources
pnpm dev:api      # FastAPI port :8000
pnpm dev:web      # React Dev port :3000
```